

# Worksheet — 1.4 Data Types, Data Structures & Boolean Algebra

Name: \_\_\_\_\_ Date: \_\_\_\_

OCR A Level Computer Science (H446) · Component 01 · 1.4.1 Data types · 1.4.2 Data structures · 1.4.3 Boolean algebra

Time: ~40 min. SHOW ALL WORKING.

## Activity 1 — Conversion drill (denary ↔ binary ↔ hex)

All binary values are **8-bit unsigned**. Fill in every blank cell. Use the place-value headers to help.

| 128 | 64 | 32 | 16 | 8 | 4 | 2 | 1 |

| # | Denary | Binary (8-bit) | Hex |
|---|--------|----------------|-----|
| 1 | 217    |                |     |
| 2 |        | 10101101       |     |
| 3 |        |                | 5A  |
| 4 | 202    |                |     |
| 5 |        | 00101111       |     |
| 6 | 190    |                |     |
| 7 |        |                | 2D  |
| 8 |        | 10110011       |     |

Working space:

## Activity 2 — Two's complement (8-bit signed, range -128 to +127)

Reminder: to find -x, invert ALL bits then ADD 1.

(a) Represent each of these denary values in 8-bit two's complement:

| Denary | 8-bit two's complement |
|--------|------------------------|
| -20    |                        |
| -100   |                        |
| -1     |                        |

Working:

**(b)** Carry out these subtractions using **A + (-B)** in 8-bit two's complement. Show A, the two's complement of B, the addition, and **discard the final carry** out of the MSB. State the denary result.

**(i) 35 - 18**

A = \_\_\_\_\_  
 -B = \_\_\_\_\_ (two's complement of 18)  
 -----  
 Sum = \_\_\_\_\_ (carry out: \_\_\_\_ → discard)

Denary result = \_\_

**(ii) 20 - 50**

A = \_\_\_\_\_  
 -B = \_\_\_\_\_ (two's complement of 50)  
 -----  
 Sum = \_\_\_\_\_

Denary result = \_\_ (Is the result negative? How does the MSB tell you?) \_\_\_\_\_

### Activity 3 — Floating-point normalisation

A floating-point number is stored as **value = mantissa × 2<sup>exponent</sup>**, with the mantissa and exponent both in **two's complement**, and the binary point assumed **after the first (sign) bit of the mantissa**.

**Normalisation rule:** a positive number's mantissa must start **0.1...**; a negative number's must start **1.0...**.

You are given the **un-normalised** positive mantissa **0.0001101** (binary point after the first bit). Normalise it.

**(a)** How many places, and in which direction, must the point move so the mantissa starts **0.1...**? \_\_\_\_\_

**(b)** Moving the point that direction — does the exponent **increase** or **decrease**? \_\_\_\_\_

**(c)** Write the normalised mantissa: \_\_\_\_\_

**(d)** Write the exponent (in denary) needed so the value is unchanged: \_\_\_\_\_

**(e)** In one sentence, why do we normalise? \_\_\_\_\_

---

## Activity 4 — Binary shifts

---

For each, give the **resulting 8-bit pattern** AND state the **operation/effect** (direction and  $\times$  or  $\div 2^n$ ). Bits shifted off the end are lost.

(a) 00010110 shifted **logical left by 2**: Result = \_\_ Effect = \_\_\_\_\_

(b) 10110000 shifted **logical right by 3**: Result = \_\_ Effect = \_\_\_\_\_

---

## Activity 5 — Data structures

---

(a) **Match** each structure (1–10) to its description (A–J). Write the letter.

| Structure      | Letter |
|----------------|--------|
| 1. Array       |        |
| 2. Record      |        |
| 3. List        |        |
| 4. Tuple       |        |
| 5. Linked list |        |
| 6. Stack       |        |
| 7. Queue       |        |
| 8. Tree        |        |
| 9. BST         |        |
| 10. Hash table |        |

**Descriptions:** - **A.** FIFO; enqueue at the rear, dequeue at the front. - **B.** Fixed-size, same-type collection accessed by index (0-based),  $O(1)$  access. - **C.** Ordered, **immutable** sequence — cannot be changed once created. - **D.** Connected, acyclic structure with a root, parents, children and leaves. - **E.** Keys mapped to indices via a hash function for  $\sim O(1)$  lookup; two keys to one index = a collision. - **F.** Nodes of data + a pointer to the next; head pointer, last points to null; no random access. - **G.** Collection of named fields that may hold **different** types. - **H.** LIFO; push/pop/peek at the top via a stack pointer. - **I.** Binary tree where the left subtree  $<$  node  $<$  right subtree. - **J.** Ordered, **mutable**, resizable collection; append/insert/remove/index.

(b) **Trace table.** Start with an **empty stack** and an **empty queue**. Apply the operations in order to BOTH structures (push/enqueue the same value, pop/dequeue when shown). Record the contents after each step and the value removed.

Operations in order: **add 7, add 3, add 9, remove, add 5, remove.**

| Step | Operation | Stack contents (top → bottom) | Stack removed | Queue contents (front → rear) | Queue removed |
|------|-----------|-------------------------------|---------------|-------------------------------|---------------|
| 1    | add 7     |                               | –             |                               | –             |
| 2    | add 3     |                               | –             |                               | –             |
| 3    | add 9     |                               | –             |                               | –             |
| 4    | remove    |                               |               |                               |               |
| 5    | add 5     |                               | –             |                               | –             |
| 6    | remove    |                               |               |                               |               |

## Activity 6 — Boolean algebra & logic

(a) Complete the truth tables.

XOR ( $Q = A \oplus B$ )

| A | B | Q |
|---|---|---|
| 0 | 0 |   |
| 0 | 1 |   |
| 1 | 0 |   |
| 1 | 1 |   |

NAND ( $Q = \neg(A \cdot B)$ )

| A | B | Q |
|---|---|---|
| 0 | 0 |   |
| 0 | 1 |   |
| 1 | 0 |   |
| 1 | 1 |   |

(b) Simplify using De Morgan's ("break the bar, change the sign — negate **each** variable and swap AND ↔ OR"). Show each step.

(i)  $\neg(A \cdot B) + B$

(ii)  $\neg(\neg A + \neg B)$

(c) **Half adder.** Complete the outputs for the two inputs A and B.

| A | B | Sum ( $A \oplus B$ ) | Carry ( $A \cdot B$ ) |
|---|---|----------------------|-----------------------|
| 0 | 0 |                      |                       |
| 0 | 1 |                      |                       |
| 1 | 0 |                      |                       |
| 1 | 1 |                      |                       |

State the Boolean expressions: **Sum** = \_\_ **Carry** = \_\_\_\_

## Challenge box

A programmer claims that the 8-bit two's complement pattern `10000000` is "negative zero, so it equals 0."

- (a) What denary value does `10000000` actually represent in 8-bit two's complement? Show your reasoning using the  $-128$  place value.
- (b) Explain why two's complement (unlike sign-and-magnitude) has **only one** representation of zero.
- (c) A colour is written `#1E90FF`. Convert the green channel `90` (hex) to denary, and the blue channel `FF` to denary. Why is hex used for colours rather than raw binary?

---

## Answer key

### Activity 1 — Conversions

| # | Denary | Binary (8-bit)        | Hex |
|---|--------|-----------------------|-----|
| 1 | 217    | <code>11011001</code> | D9  |
| 2 | 173    | <code>10101101</code> | AD  |
| 3 | 90     | <code>01011010</code> | 5A  |
| 4 | 202    | <code>11001010</code> | CA  |
| 5 | 47     | <code>00101111</code> | 2F  |
| 6 | 190    | <code>10111110</code> | BE  |
| 7 | 45     | <code>00101101</code> | 2D  |
| 8 | 179    | <code>10110011</code> | B3  |

Worked examples: - **217**:  $128+64=192$ ,  $+16=208$ ,  $+8=216$ ,  $+1=217 \rightarrow 11011001$ . Nibbles `1101 1001` = D9. - **10101101**:  $128+32+8+4+1 = 173$ . Nibbles `1010 1101` = AD. - **5A**:  $5=0101$ ,  $A=1010 \rightarrow 01011010 =$

64+16+8+2 = 90. - **202**: 128+64=192, +8=200, +2=202 → 11001010 = 1100 1010 = CA. - **00101111**: 32+8+4+2+1 = 47. 0010 1111 = 2F. - **190**: 128+32=160, +16=176, +8=184, +4=188, +2=190 → 10111110 = BE. - **2D**: 2=0010, D=1101 → 00101101 = 32+8+4+1 = 45. - **10110011**: 128+32+16+2+1 = 179. 1011 0011 = B3.

## Activity 2 — Two's complement

(a)

| Denary | 8-bit two's complement | Working                                         |
|--------|------------------------|-------------------------------------------------|
| -20    | <b>11101100</b>        | +20 = 00010100; invert 11101011; +1 = 11101100  |
| -100   | <b>10011100</b>        | +100 = 01100100; invert 10011011; +1 = 10011100 |
| -1     | <b>11111111</b>        | +1 = 00000001; invert 11111110; +1 = 11111111   |

Check -20: -128+64+32+8+4 = -20 ✓. -100: -128+16+8+4 = -100 ✓.

(b)(i) 35 - 18 = 17

```

A   = 00100011   (35)
-B  = 11101110   (-18: +18=00010010 → invert 11101101 → +1 = 11101110)
-----
      1 00010001   carry out of MSB = 1 → DISCARD
Sum  = 00010001

```

Denary result = +17 (MSB 0 = positive; 16+1 = 17) ✓

(b)(ii) 20 - 50 = -30

```

A   = 00010100   (20)
-B  = 11001110   (-50: +50=00110010 → invert 11001101 → +1 = 11001110)
-----
Sum  = 11100010   (no carry out of MSB)

```

Denary result = -30. The MSB is 1, so the result is negative. Check: -128+64+32+2 = -30 ✓.

## Activity 3 — Floating-point normalisation

Given mantissa 0.0001101 (point after first bit). - (a) Move the point **3 places to the right** so the mantissa becomes 0.1101... (first bit after the point is 1). - (b) Moving the point **right** ⇒ **exponent decreases**. - (c) Normalised mantissa = **0.1101000** (0.1101 with trailing zero padding). - (d) Original exponent was effectively 0; moving right 3 places means we must **subtract 3** → exponent = **-3**. (Value = 0.0001101 × 2<sup>0</sup> = 0.1101 × 2<sup>-3</sup>, unchanged.) - (e) Normalisation removes wasted leading zeros so the fixed-width mantissa holds the **maximum number of significant figures** → **greatest precision** (and gives a unique representation).

## Activity 4 — Binary shifts

(a) 00010110 logical left 2 = **01011000**. Effect: shift **left**, multiplies by 2<sup>2</sup> = **×4**. (Check: 22 × 4 = 88 = 01011000 ✓.)

(b) 10110000 logical right 3 = 00010110 . Effect: shift **right**, divides by  $2^3 = \div 8$  (integer division; bits shifted off are lost, 0s filled from the left). (Check:  $176 \div 8 = 22 = 00010110$  ✓.)

## Activity 5 — Data structures

(a) 1 → B, 2 → G, 3 → J, 4 → C, 5 → F, 6 → H, 7 → A, 8 → D, 9 → I, 10 → E.

(b) Trace table (stack = LIFO, removes the **top/most recent**; queue = FIFO, removes the **front/oldest**):

| Step | Operation | Stack (top → bottom) | Stack removed | Queue (front → rear) | Queue removed |
|------|-----------|----------------------|---------------|----------------------|---------------|
| 1    | add 7     | 7                    | —             | 7                    | —             |
| 2    | add 3     | 3, 7                 | —             | 7, 3                 | —             |
| 3    | add 9     | 9, 3, 7              | —             | 7, 3, 9              | —             |
| 4    | remove    | 3, 7                 | <b>9</b>      | 3, 9                 | <b>7</b>      |
| 5    | add 5     | 5, 3, 7              | —             | 3, 9, 5              | —             |
| 6    | remove    | 3, 7                 | <b>5</b>      | 9, 5                 | <b>3</b>      |

## Activity 6 — Boolean

(a) XOR

| A | B | Q        |
|---|---|----------|
| 0 | 0 | <b>0</b> |
| 0 | 1 | <b>1</b> |
| 1 | 0 | <b>1</b> |
| 1 | 1 | <b>0</b> |

NAND

| A | B | Q        |
|---|---|----------|
| 0 | 0 | <b>1</b> |
| 0 | 1 | <b>1</b> |
| 1 | 0 | <b>1</b> |
| 1 | 1 | <b>0</b> |

(b)(i)  $\neg(A \cdot B) + B = (\neg A + \neg B) + B$  (De Morgan's on the bracket) =  $\neg A + (\neg B + B)$  (associative/commutative) =  $\neg A + 1$  (complement:  $\neg B + B = 1$ ) = **1** (null law). The expression is always true.

(b)(ii)  $\neg(\neg A + \neg B) = \neg(\neg A) \cdot \neg(\neg B)$  (De Morgan's: break the bar, swap OR → AND) =  $A \cdot B$  (double negation on each term) = **A · B**.

(c) Half adder

| A | B | Sum | Carry |
|---|---|-----|-------|
| 0 | 0 | 0   | 0     |
| 0 | 1 | 1   | 0     |
| 1 | 0 | 1   | 0     |
| 1 | 1 | 0   | 1     |

**Sum =  $A \oplus B$  Carry =  $A \cdot B$ .**

### Challenge box

**(a)**  $10000000$  = the  $-128$  column only set, all others 0  $\rightarrow$   **$-128$** . (It is not zero; zero is  $00000000$ .)

**(b)** In two's complement the MSB is a single negative weight, so each bit pattern maps to exactly one value;  $00000000$  is the only pattern that sums to 0. Sign-and-magnitude has both  $00000000$  (+0) and  $10000000$  ( $-0$ ); two's complement avoids this, giving **one** zero (and one extra negative value,  $-128$ ).

**(c)** Green  $90_{16} = (9 \times 16) + 0 = \mathbf{144}$ . Blue  $FF_{16} = (15 \times 16) + 15 = \mathbf{255}$ . Hex is used because each pair of hex digits = exactly one byte (8 bits / one colour channel), making it a **compact, human-readable, lossless** shorthand for the underlying binary, far less error-prone than writing 24 raw bits.